

计算机系统能力实训第一周周记

实训题目：Windows 命令行解释器设计

本周主题：任务分析、资料查阅、环境搭建与系统初步设计

1 本周实训任务

本周是计算机系统能力实训的第一周，实训题目为“Windows 命令行解释器设计”。根据任务书要求，本周主要围绕任务理解、资料查阅、Windows API 学习、开发环境搭建和系统初步设计展开，为后续编码实现打基础。

通过阅读任务书，我明确本次实训并不是简单编写一个普通控制台程序，而是要结合操作系统、计算机组成原理、编译原理和系统调用等课程知识，设计并实现一个具有系统软件特征的命令行解释器。该解释器需要参考 Windows 的 Command 命令解释程序，采用控制台命令行输入方式，显示当前目录和提示符，并能够执行常见内部命令。

第一周的核心目标可以概括为：理解项目背景，明确需求边界，搭建开发环境，整理关键 API，并完成 WinShellX 的初步总体设计。

2 本周完成的主要工作

2.1 任务书分析

本周首先对实训任务书进行了详细阅读。任务书要求设计一个 Windows 命令行解释器，能够通过控制台读取用户输入，并执行常见内部命令，例如 `cd`、`dir`、`history`、`exit`、`tasklist`、`taskkill` 等。

通过分析任务书，我将本项目定位为一个教学型命令解释器。它不需要完全复刻 Windows 的 `cmd.exe`，但需要体现命令解释程序的基本工作过程，包括命令输入、命令解析、参数处理、功能分发、系统 API 调用、错误处理和结果输出。

根据任务要求，本项目采用 C++ 作为主要开发语言，并通过 Win32 API 调用 Windows 系统功能。命令解释器运行在 Windows 控制台中，用户在提示符后输入命令，程序根据命令名称调用相应模块完成操作。

本周初步确定项目名称为 **WinShellX**。其中，Win 表示运行平台为 Windows，Shell 表示命令行解释器，X 表示系统具有可扩展能力，后续可以继续添加外部命令执行、历史命令持久化、智能补全、管道重定向等扩展功能。

2.2 命令解释程序与内核关系学习

本周学习了命令解释程序在操作系统中的位置。命令解释程序不是操作系统内核本身，而是用户与操作系统之间的接口程序。用户输入命令后，命令解释器负责识别命令、解析参数，并通过系统调用或操作系统提供的 API 请求内核完成实际操作。

例如，用户输入 `dir` 时，命令解释器需要调用文件系统相关接口遍历目录内容；用户输入 `tasklist` 时，需要调用进程枚举相关接口获取系统进程信息；用户输入 `taskkill` 时，需要通过进程控制接口打开并终止指定进程。

通过资料学习，我进一步理解了硬件、内核、系统调用和命令解释程序之间的层次关系。硬件提供最底层的计算和存储能力；操作系统内核负责资源管理；系统调用或 Windows API 为应用程

序提供访问系统功能的接口；命令解释器则处于用户和系统 API 之间，负责将用户命令转化为具体的系统调用过程。

这一部分学习使我认识到，命令解释器虽然表面上只是一个控制台交互程序，但它实际上连接了用户输入、操作系统 API 和底层资源管理，是理解操作系统工作方式的一个很好的实践入口。

2.3 Windows API 学习

针对 `cd` 和 `dir` 命令，本周重点学习了目录和文件操作相关 API。

API 名称	学习内容
<code>GetCurrentDirectory</code>	获取当前工作目录，用于显示命令提示符和实现 <code>cd</code> 无参数查询。
<code>SetCurrentDirectory</code>	设置当前工作目录，用于实现 <code>cd < 路径 ></code> 。
<code>FindFirstFile</code>	查找目录中的第一个文件或子目录，用于实现 <code>dir</code> 。
<code>FindNextFile</code>	继续遍历目录中的后续文件或子目录。
<code>FindClose</code>	关闭文件查找句柄，避免资源泄漏。

通过学习这些 API，我理解了 Windows 中目录遍历并不是一次性返回全部结果，而是通过查找句柄逐项获取目录项。

针对 `tasklist` 和 `taskkill` 命令，本周整理了进程管理相关 API。

API 名称	学习内容
<code>CreateToolhelp32Snapshot</code>	创建系统进程快照，用于获取当前进程列表。
<code>Process32First</code>	获取快照中的第一个进程信息。
<code>Process32Next</code>	遍历快照中的后续进程信息。
<code>OpenProcess</code>	根据 PID 打开目标进程，获取进程句柄。
<code>TerminateProcess</code>	终止指定进程。
<code>CloseHandle</code>	关闭进程、快照等系统对象句柄。

这一部分学习让我认识到，进程管理功能涉及系统权限和句柄管理。对于系统进程或权限不足的进程，程序可能无法成功打开或终止，因此后续实现时必须做好错误处理。

除了文件和进程 API，本周还初步了解了控制台相关 API。控制台程序可以使用标准输入输出完成基本交互，也可以使用 Console API 完成清屏、彩色输出和按键读取等增强功能。

这些内容为后续实现命令提示符、历史命令浏览、智能补全和彩色输出等功能提供了技术基础。

2.4 开发环境搭建

本周完成了 Windows C/C++ 开发环境配置。开发工具采用 Visual Studio C++ 工具链，并在命令行中使用 `cl` 编译器进行测试。

环境验证时编写了一个简单 C++ 程序，并使用以下命令编译运行：

```
cl /EHsc main.cpp
.\main.exe
```

程序成功输出测试信息，说明编译器、链接器和运行环境可以正常使用。通过这一步，我确认后续可以直接在本地进行 WinShellX 的开发和测试。

第一周阶段先以 `cl` 命令直接编译为主，便于快速验证代码。后续计划加入 `CMakeLists.txt`，使项目能够按规范方式组织多个源文件，并方便后续扩展模块和统一构建。

2.5 系统初步设计

本周初步设计了 WinShellX 的主流程。程序启动后进入循环，每次循环完成以下步骤：

- 1. 获取当前工作目录。
- 2. 显示当前目录和提示符。
- 3. 读取用户输入的一整行命令。
- 4. 去除多余空白并保存历史记录。
- 5. 解析命令名和参数。
- 6. 查询命令注册表。
- 7. 调用对应命令处理函数。
- 8. 输出结果或错误提示。
- 9. 遇到 `exit` 时结束程序。

这一流程基本覆盖了命令解释器的核心工作过程。
为了避免所有代码集中在一个文件中，本周对项目模块进行了初步划分。

模块名称	职责说明
Shell 主控模块	负责主循环、提示符显示、命令读取和命令分发。
命令解析模块	负责将用户输入解析为命令名和参数列表。
命令注册模块	负责维护命令名称与命令处理函数之间的映射关系。
内置命令模块	负责实现 <code>cd</code> 、 <code>dir</code> 、 <code>history</code> 、 <code>exit</code> 、 <code>tasklist</code> 、 <code>taskkill</code> 等命令。
工具模块	负责字符串处理、路径处理、WinAPI 错误信息转换等通用功能。
测试模块	负责编写手工测试用例，记录测试输入、预期结果和实际结果。

在设计时，我重点考虑高内聚、低耦合原则。主循环只负责流程控制，不直接实现具体命令；每个命令尽量封装成独立函数或独立模块；通用逻辑放入工具模块，避免重复代码。

这样做的好处是后续新增命令时不需要大幅修改主循环，只需要实现新的命令处理函数并注册到命令表中即可。

3 本周遇到的问题及解决办法

3.1 Windows API 参数复杂

刚开始学习 Win32 API 时，很多函数参数较多，并且涉及结构体、指针、句柄和返回值检查。例如进程枚举函数需要先创建快照，再通过结构体逐项读取进程信息；进程终止还需要先打开进程句柄，再调用终止函数。

我采用由简单到复杂的方式学习。先理解 `GetCurrentDirectory`、`SetCurrentDirectory` 这类较简单的 API，再学习文件遍历和进程管理 API。同时在整理 API 时记录函数用途、关键参数、返回值和可能出现的错误，为后续编码提供参考。

3.2 功能边界需要控制

Windows 自带的 `cmd.exe` 功能非常复杂，包含内部命令、外部命令、批处理、环境变量、管道、重定向等大量功能。如果第一阶段就追求完全复刻，工作量会过大，也容易影响核心功能完成。

本周将项目功能划分为基础功能和扩展功能。基础功能优先满足任务书要求，包括 `cd`、`dir`、`history`、`exit`、`tasklist`、`taskkill`。扩展功能作为后续加分点逐步实现，例如外部命令执行、历史持久化、智能补全、管道重定向等。

3.3 项目结构需要提前设计

如果所有代码都写在 `main.cpp` 中，虽然短期内实现速度较快，但随着功能增加，代码会变得难以维护。

本周提前规划了项目目录结构，计划使用 `include/` 存放头文件，`src/` 存放源文件，`commands/` 存放命令实现，`utils/` 存放通用工具，`tests/` 存放测试用例。这样可以保证后续开发过程更清晰，也更符合课程设计对工程规范的要求。

4 本周收获

4.1 对命令解释器有了更具体的认识

通过第一周学习，我认识到命令解释器并不是简单读取字符串再输出结果，而是一个连接用户输入和操作系统 API 的系统软件。它需要完成命令识别、参数解析、系统调用、错误处理和输出展示等多个环节。

4.2 加深了对 Windows API 的理解

通过查阅和整理 API，我理解了应用程序如何通过 Win32 API 请求操作系统完成具体工作。例如目录切换、文件遍历、进程枚举和进程终止都需要调用不同类别的 API。这让我对操作系统课程中“系统调用”的概念有了更直观的理解。

4.3 明确了后续开发路线

本周完成了项目名称、技术路线、模块划分和开发计划的初步确定。后续可以按照“先实现基础框架，再实现内部命令，最后增加扩展功能”的顺序推进。

5 下周计划

5.1 完成基础框架

下周计划首先搭建 WinShellX 的基础代码框架，实现主函数、Shell 主循环、提示符显示和命令读取。

5.2 实现命令解析与注册

计划实现命令解析模块和命令注册模块，使程序能够根据用户输入找到对应命令处理函数，为后续新增命令打基础。

5.3 实现基础内部命令

下周优先实现以下命令：

1. `cd`：切换和显示当前目录。
2. `dir`：显示目录内容。
3. `history`：显示历史命令。
4. `exit`：退出程序。

5.4 开始进程命令实现

在基础命令完成后，计划开始实现 `tasklist` 和 `taskkill`，并测试进程枚举和进程终止功能。

6 本周小结

第一周主要完成了任务理解、资料查阅、开发环境搭建、Windows API 初步学习和系统总体设计。通过本周工作，我明确了 WinShellX 的开发目标和技术路线，也认识到命令解释器虽然交互形式简单，但其内部涉及操作系统接口、命令解析、进程管理、文件系统访问等多个系统软件知识点。

总体来看，本周工作为后续编码实现奠定了基础。接下来将按照模块化设计逐步完成程序实现和测试，并在满足任务书基本要求的基础上继续完善扩展功能。